



Retail Behavior Monitoring Through Action Detection Using Computer Vision

Computer Science Department, FPT University, Hanoi, Vietnam

Vu Quang Hai
Le Quang Huy
Bach Minh Nam
Trinh Quoc Dat
Pham Hoang Phuong

Supervisor: Bui Van Hieu

Bachelor of Artificial Intelligence
Hoa Lac campus – FPT University
2025

Acknowledgment

We would hereby like to express great thanks to Supervisor Bui Van Hieu for his great support and his good counsel during the past four months. His good guidance and encouragement allowed us to refine our ideas and previous difficulties in our research. Without him, we were not able to complete this thesis successfully.

We are also deeply grateful to all the lecturers at FPT University, particularly those in the Information Technology Specialization Department. The knowledge and skills acquired from their classes laid the essential foundation for our research endeavors. Their dedication to teaching and commitment to student success have been a constant source of inspiration.

Finally, we would also like to acknowledge the appreciation of our families and friends, who were consistent sources of inspiration and motivation. Their words of encouragement and valuable advice motivated us to stay focused and never give up. We are enormously grateful for their support and affection, which allowed us to complete this thesis.

Abstract

Customer shopping behavior is one of the most important variables that enhance user experience and business efficiency within the retail sector. With increasing applications of computer vision techniques in retail behavior analysis, action detection within retail scenes automatically has been an active area of research. Besides, this task remains challenging due to the subtle nature of customer movements, visual similarity between actions, occlusions, and inconsistent lighting conditions. In this experimental investigation, we evaluate the effectiveness of a multi-stage action detection framework designed for retail surveillance. The framework initially leverages human pose estimation to obtain robust structural cues and applies MViT together with hierarchical attention mechanisms to extract temporal patterns. A RNN Post-Processing module is introduced to enhance temporal coherence and reduce prediction fragmentation. Evaluated on the MERL Shopping dataset, the evaluated framework achieved frame-wise accuracy of 88.4%, segment-level F1@50 of 65.2% and mAP@midpoint of 95.7%.

Keywords: Retail Surveillance; Pose Estimation; Vision Transformer; RNN Post-Processing

Table of Contents

1	Introduction	8
2	Related Work	10
2.1	Retail Action Recognition	10
2.2	Multi-Modal Fusion	11
2.3	Temporal Post-Processing	11
3	Methodology	12
3.1	RGB Stream: Visual Feature Extraction	13
3.1.1	Global Context Path	14
3.1.2	Local Detail Path	14
3.1.3	Projection and Gating	15
3.2	Pose Stream: Modeling Pose Dynamics	16
3.3	Multi-Modal Fusion and Temporal Refinement	18
3.4	Prediction heads	19
3.5	RNN-based Temporal Post-Processing	20
4	Experiments and Results	22
4.1	Dataset and Experimental Setup	22
4.2	Frame Preprocessing and Window Sampling	23
4.3	Implementation Details	24
4.3.1	Base Model Training	24
4.3.2	RNN Input Data Generation	25
4.3.3	RNN Post-Processor Training	26

4.4	Evaluation Metrics	27
4.5	Evaluation of Architectural Components and Fusion Strategies	29
4.6	Results and Analysis	30
4.7	Discussion	31
5	Conclusion	33
References		34

List of Tables

Table 1:	Action segment counts and total durations in the MERL Shopping Dataset [27]	23
Table 2:	Action-duration statistics (frames)	23
Table 3:	Comparative performance results over three experiments . .	30
Table 4:	Benchmark comparison on the MERL Shopping Dataset . .	31

List of Figures

Figure 1: Overview of the evaluated multi-stream temporal action-detection architecture	12
Figure 2: Detailed diagram of the RGB-stream feature extractor . . .	13
Figure 3: Pose-dynamics sub-network	16
Figure 4: Multi-modal fusion module	18
Figure 5: Diagram Showing the Prediction Heads	20
Figure 6: Overview of the RNN-Based Temporal Post-Processing Module	21
Figure 7: Images for Various Actions in the MERL Shopping Dataset [27]	22

List of Abbreviations and Acronyms

AP:	Average Precision
ASRF:	Action Segment Refinement Framework
AMFI-Net:	Attention-Based Multimodal Feature Integration
Bi-LSTM:	Bidirectional Long Short-Term Memory
CNN:	Convolutional Neural Network
CRF:	Conditional Random Fields
ED-TCN:	Encoder-Decoder Temporal Convolutional Network
F1:	F-score (harmonic mean of precision and recall)
HA-Net:	Hierarchical Attention Network
HaDa:	Hands Dataset
IoU:	Intersection over Union
I3D:	Inflated 3D ConvNet
LSTM:	Long Short-Term Memory
MLP:	Multi-Layer Perceptron
MViT:	Multiscale Vision Transformer
MS-TCN:	Multi-Stage Temporal Convolutional Network
MS-BRNN:	Multi-Stream Bi-Directional Recurrent Neural Network
mAP:	mean Average Precision
NTU RGB+D:	Nanyang Technological University RGB+Depth Dataset
RNN:	Recurrent Neural Network
ResNet:	Residual Network
RGB:	Red-Green-Blue
TCN:	Temporal Convolutional Network
JMRN:	Joint Motion and Relation Network

1 Introduction

Monitoring retail behavior is increasingly crucial in managing store operations more efficiently, delivering better customer experience, and pursuing a competitive edge for businesses. Retailers today face immense pressure to deliver customized and cost-effective shopping experiences with minimum operating expenses. Current retail surveys emphasize the rapid take-up of AI in retail: an NVIDIA report indicates that 89% of retailers globally have adopted or piloted AI solutions [5], and 63% consider AI a key driver of long-term growth [4]. In addition, 78% of the leading retail chains have invested in AI-backed customer behavior analysis [28]. In real-deployment environments, Walmart experienced a 15% shrinkage reduction with AI-based behavior observation [12], and Amazon Go’s fully automated, checkout-free stores have lowered operating costs by 25–30% [10]. These figures indicate the strategic importance and tangible impact of AI in modern retail, motivating our investigation into the empirical performance of established multimodal action-detection architectures for real-time store monitoring. This statistic not only demonstrates the widespread take-up of AI throughout retail activity but also shows the near-universal acceptance of AI, indicating the role of the technology in facilitating personalization, enhancing operational efficiency, and controlling costs.

For instance, computer vision-based action recognition systems enable real-time monitoring of customers and thereby enable personalized customer experience and efficient allocation of resources [26]. Pantano & Vannucci [22] and Dell Technologies [6] case studies illustrate how AI-powered monitoring improves sales, customer retention, and the overall shopping experience. In general, advanced computer vision-based retail monitoring solutions offer scalable, efficient insights for addressing personalization needs, enhancing operational efficiency, and ensuring cost containment.

Despite these promising advancements, retail store settings are highly demanding for behavior monitoring. The majority of retail stores employ stationary, ceiling-mounted cameras at elevated angles, reducing the field of view and leading to occlusions due to shelving, displays, or other shoppers. Such fixed viewpoints provide only partial body motion and suffer from perspective distortions, thereby rendering effective action recognition challeng-

ing. Also, variable shelf levels and confined aisles further constrain spatial accessibility, disallowing complete observation of intricate hand–object interactions from many perspectives [29]. Shoppers motions in this context tend to be extremely brief, inconspicuous, and visually ambiguous, necessitating rich temporal positioning and fine-grained differentiation. Reaching for a product, retracting the hand after contact, or inspecting a held product can all appear nearly identical in typical video streams. This subtlety and overlap of movements underscores why advanced AI and computer vision techniques are essential: only through advanced spatio–temporal modeling and robust feature extraction can retailers accurately detect, classify, and respond to these behaviors in real time [27].

In this experiment, we use the MERL Shopping Dataset [27] as our primary benchmark for retail action detection. While prior action-recognition research yields valuable insights, those evaluated on MERL nonetheless exhibit fundamental shortcomings: the Two-Stream CNN proposed by Simonyan & Zisserman [26] employs externally computed optical flow, which is computationally costly and bars end-to-end learning; TCN introduced by Lea et al. [17] are low latency but possess a fixed receptive field, decouple spatial feature extraction and temporal modeling, and fare poorly with occlusions or non-uniform lighting; the Multi-Stream Bi-Directional RNN developed by Singh et al. [27] combines multiple CNN and Bi-LSTM streams to enhance accuracy but entails large compute, tends to over-segment actions in noisy or complex motion instances; attention-based methods weakly-supervised framework by Weinzaepfel et al. [31] and hierarchical attention networks such as HA-Net by Gammulle et al. [11], while reducing segmentation errors, significantly increase model complexity and inference time while still failing to exploit structural pose and velocity cues, shortcomings that hamper robustness and fine-grained action discrimination.

In this experimental study, we focus on empirically assessing established multimodal architectures for retail-action detection rather than proposing a new model. Our benchmark pipeline combines global appearance cues captured by a pre-trained MViT, fine-grained spatial details extracted by a lightweight ResNet-18 path, and motion plus skeletal dynamics encoded by a Bi-LSTM that processes pose sequences. After late fusion of these three streams, a lightweight TCN further refines the sequence before classification, and a Bi-LSTM post-processing block smooths per-frame predictions and sharpens action boundaries. We also investigate how different fusion strategies (concatenation versus weighted sum) and class-balanced loss weighting affect mAP@midpoint and F1@50 on the MERL Shopping dataset.

2 Related Work

To contextualize our experimental approach within the landscape of retail video action analysis, we first review existing research aligned with the core stages of our pipeline: Retail Action Recognition (RGB and motion modeling), Multi-Modal Fusion (adaptive integration of RGB, pose, depth, and trajectory), and Temporal Post-Processing (long-range convolutional refinement and RNN-based smoothing). We present representative works in each category and highlight how their insights inform the design of our unified framework.

2.1 Retail Action Recognition

Research on retail-specific action recognition has leveraged deep video models to address challenges such as overhead/top-down viewpoints, occlusion from shelves, and uneven lighting. Singh et al. [27] introduced the MERL Shopping Dataset and a multi-stream Bi-LSTM (MS-BRNN) that fused full-frame and person-centered RGB + optical-flow features to yield contiguous action segments (e.g., Reach-to-Shelf, Inspect Product), outperforming frame-wise two-stream CNN baselines in F1 and mAP metrics. Building on general video backbones, Carreira & Zisserman’s I3D [3] inflated 2-dimensional CNN filters into 3-dimensional, enabling strong spatio-temporal feature learning, while Feichtenhofer et al.’s SlowFast [9] introduced dual branches, a slow pathway for spatial context and a fast pathway for motion, achieving state-of-the-art results on benchmarks and showing promise when fine-tuned on MERL. More recently, Fan et al.’s MViT [7] applied a hierarchical multiscale transformer to videos, achieving robust recognition of subtle customer, product interactions under occlusions and clutter by progressively aggregating coarse-to-fine temporal features. Other specialized systems, such as Paolanti et al. [23], combined overhead RGB-D input with parallel CNNs to classify shopper interactions in real time, achieving high accuracy on the HaDa dataset, and Wen et al. [32] demonstrated that depth and multi-view aggregation help mitigate lighting and background variation.

2.2 Multi-Modal Fusion

To disambiguate visually similar retail actions, many works fuse complementary modalities (RGB, pose, optical flow, depth, trajectory). On MERL, Singh et al. [27] demonstrated that fusing appearance and motion streams via a Bi-LSTM reduced fragmentation compared to single-modality models. Moghaddam et al. [21] extended this by incorporating noisy pose estimation and object-centric motion into an attention-based fusion module, adaptively weighting pose versus RGB features to distinguish fine-grained gestures (e.g., pick vs. put) and achieved state-of-the-art accuracy on MERL Shopping [27]. Depth-enhanced fusion has also been effective: Paolanti et al. [23] used synchronized RGB + depth streams to improve interaction classification, especially under occlusion. Recent transformer-based fusion, such as Zhao et al.’s AMFI-Net [34], jointly processes skeleton and RGB inputs, employing confidence-weighted attention to gate modalities dynamically; this achieved significant gains on NTU RGB+D, illustrating that gating by pose reliability enhances robustness when pose is noisy. Late-fusion strategies also prevail: Shah et al. [25] demonstrated that combining a pose-based action recognizer JMRN with an RGB + flow CNN in a late fusion step yields complementary improvements, underscoring that pose and appearance provide orthogonal cues in retail behavior. Overall, these studies show that effective multi-modal fusion, particularly attention-based or confidence-aware schemes, yields more accurate and robust fine-grained action recognition in retail settings while balancing model complexity.

2.3 Temporal Post-Processing

Since behavior in store entails several frames of behavior, post-processing over time is needed to generate consistent labels and accurate segment edges. Singh et al. [27] applied a Bi-LSTM to frame-wise outputs of CNNs to smooth and reduce false splits of MERL video segments. Temporal Convolutional Networks, then, offered a convolutional solution: Lea et al. [17] offered ED-TCN, which employs encoder-decoder 1-dimensional convolution and pooling/upsampling to capture long-range relationships and achieved substantial F1 gains over RNNs on fine-grained segmentation benchmarks. Farha & Gall’s MS-TCN [8] stacked stages of refinement of TCNs, building on ED-TCN over benchmark datasets; MS-TCN++ of Li et al. [18] further improved using dual dilated convolution and an optimized refinement architecture, achieving competitive F1 scores on tasks like 50 Salads. Boundary-aware

methods have since emerged: Ishikawa et al. [15] proposed ASRF, which outputs action class and explicit likelihoods of edges, then post-processes frame labels to merge or sub-divide segments from estimated edges and avoid over-segmentation. Most recently, Yi et al. [33] proposed ASFormer, a transformer model based on local self-attention windows and hierarchical encodings to process long-term context effectively; ASFormer improved variants of TCNs by several percentage points over benchmarks like Breakfast and 50 Salads for F1 and IoU scores. Such advances show that multi-stage convolutional refinement, boundary regression, and modeling by attention significantly boost segmental accuracy for continuous action detection and provide the temporal coherence needed for trustworthy observation of behavior in store.

3 Methodology

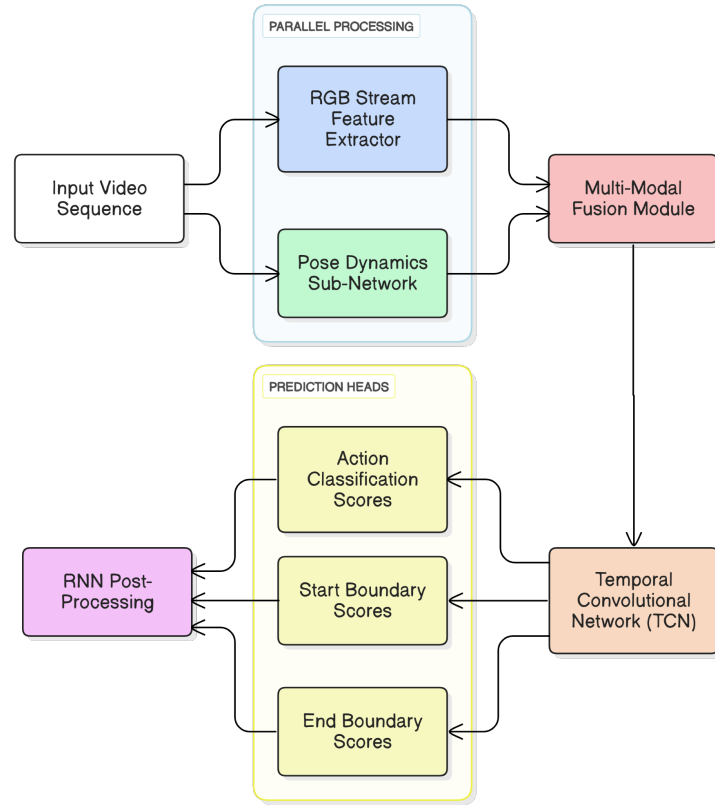


Figure 1: Overview of the evaluated multi-stream temporal action-detection architecture

Temporal action detection from untrimmed videos must cope with actions of highly variable lengths, complex background clutter, and the capacity to resolve high-frequency motion cues [35]. Trying to overcome these, we present an end-to-end pipeline known as MViT and Pose Network for fine grained retail behavior monitoring (show in Figure 1). We start by segmenting each raw video into overlapping clips of $T = 32$ frames, sampled with 50% overlap (explained in 4.2). In both videos, a per-video-specific MViT extracts global spatio-temporal features and the two-layer Bi-LSTM extracts structural motion from concatenated pose and frame-to-frame velocity vectors. The two streams are concatenated at each transformer block and also scrubbed with a light TCN to extract modalities and time relations. Finally, three 1-dimensional convolution heads predict per-frame action, start and end boundary logits. This rich representation, global context, fine local detail, and explicit pose dynamics hierarchy can achieve precise temporal localization in real-world retail applications. After training, we apply an auxiliary RNN smoothing module with dropout and layer normalization to minimize isolated peaks, leading to temporally coherent segments.

3.1 RGB Stream: Visual Feature Extraction

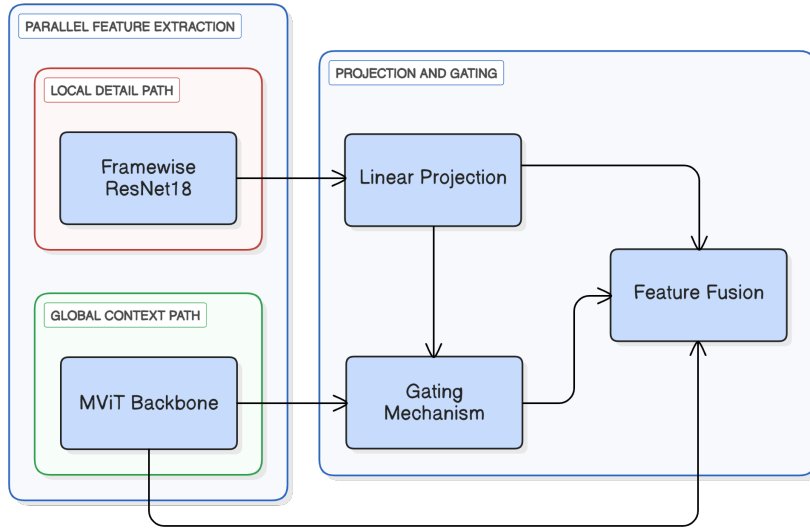


Figure 2: Detailed diagram of the RGB-stream feature extractor

3.1.1 Global Context Path

The visual stream is crucial to record appearance information and scene context, applying a hybrid technique that identifies actions conveyed by total scene changes in addition to single object manipulations or movements. It applies spatio-temporal feature extraction to global context together with frame-level analysis for local detail, then followed by targeted temporal refinement and a dynamic integration mechanism. As indicated by Figure 2, the RGB-stream feature extractor is composed of two complementary paths. The local-detail path first processes using a frame-wise ResNet-18 backbone. In parallel, the global-context path employs the MViT-S backbone for encoding multi-scale spatio-temporal features for the T frames clip. The outputs of the two streams are linearly projected onto the same dimensionality and merged through an adaptive gating mechanism that balances the contribution of each stream based on the input material. The merged feature vector is then fed into the fusion module for inter-stream fusion.

To account for long-range spatio-temporal relations across the whole input clip, a MViT backbone is utilized as the primary backbone [7]. MViT, building on the triumph of transformers within computer vision and natural language processing, are well-suited to video because they are good at modeling relations in both spatial and temporal spaces via attention mechanisms. Unlike traditional 3-dimensional CNNs, which rely on fixed receptive fields, transformers can dynamically pay attention to relevant information across the whole clip and are thus appropriate for capturing complex actions that unfold over long time periods [1, 30]. This specific MViT variant takes a multi-stage block architecture with varying attention mechanisms and channel widths, processing video clips of standard size ($3 \times T \times H \times W$), where T is the temporal window duration, and H/W are height/width). The resulting output is a general global feature description of the clip, of dimension $D_{\text{global}} = 768$, which is meant to capture the global scene dynamics and action context of the temporal window.

3.1.2 Local Detail Path

Complementary to the global view, fine-grained within-frame level detailed information is achieved using a pre-trained 2-dimensional Convolutional Neural Network. A specific ResNet-18 architecture [13], which is pre-trained on an enormous dataset like ImageNet in advance, is utilized. Such

pre-trained weights (transfer learning) are applied, which accelerate convergence significantly and improve performance especially when sparse labeled video data exist. The final classification layer of ResNet-18 is removed so that the network operates as a feature extractor, providing accurate frame-level spatial feature vectors of dimension $D_{\text{local}} = 512$. The operation is performed individually on every frame in the temporal window T and provides a series of local feature vectors $\mathbf{F}_{\text{local}} \in \mathbb{R}^{B \times T \times D_{\text{local}}}$ (where B is batch size). Of these, critical is the weight freezing of this 2-dimensional CNN during training. This computation reduces and preserves pre-trained features from being drastically altered, with their robust general-purpose representational capacity optimized for static image comprehension that serves as a good foundation for frame-level detail.

3.1.3 Projection and Gating

To enable efficient combination, each 512-dimensional detailed local feature vector is mapped into the 768-dimensional global embedding space by a linear projection layer, yielding $\mathbf{L}' \in \mathbb{R}^{B \times T \times 768}$ salient aspect of this architecture is the dynamic combination of the global context ($\mathbf{G} \in \mathbb{R}^{B \times D_{\text{local}}}$, temporally expanded) and these mapped local features $\mathbf{L}'$. Observing that the relative importance of the global context and the local detail may vary with the specific action and time step, a gating mechanism based on attention principles is suggested [30]. Such a gate calculates adaptive weights W_g from the concatenated global and local information. At every time step, $\mathbf{L}'$ and a copy of $\mathbf{G}$ separated from it are concatenated into a 1536-dimensional vector and passed through a two-layer MLP ($1536 \rightarrow 384 \rightarrow 768$) with ReLU activation and dropout followed by a Sigmoid. This produces a per-channel gate weight in $[0,1]$ that adaptively trades off between local and global information. The final RGB stream feature representation $\mathbf{F}_{\text{rgb}} \in \mathbb{R}^{B \times T \times D_{\text{global}}}$ is then computed through a weighted combination, mathematically expressed as

$$\mathbf{F}_{\text{rgb}} = W_g \odot \mathbf{L}' + (1 - W_g) \odot \mathbf{G}, \quad (1)$$

where $\odot$ represents element-wise multiplication. This allows the model to selectively concentrate on either frame-level details or the overall context at each time step. We keep $\mathbf{G}$ distinct before the gate so that during backpropagation, the model updates primarily local projection and gate parameters,

training becoming more stable and feature specialization improved.

3.2 Pose Stream: Modeling Pose Dynamics

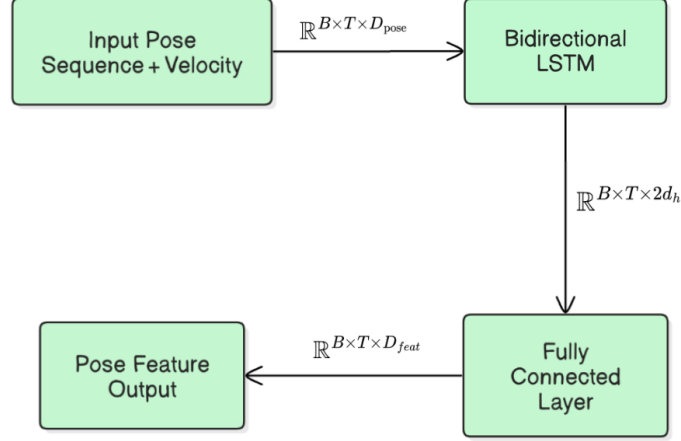


Figure 3: Pose-dynamics sub-network

While RGB features provide appearance information, skeletal keypoints offer robust structural and motion cues. The pose stream, responsible for modeling pose dynamics, employs a sub-network visualized in Figure 3. This sub-network processes 3-dimensional keypoint sequences; in this experiment, these sequences are extracted using MediaPipe [20].

The input to this sub-network at each frame t is a feature vector $\mathbf{x}_t$. This vector is formed by concatenating a 99-dimensional vector of spatial keypoint coordinates, $\mathbf{p}_t$, at frame t with its corresponding 99-dimensional vector of frame-to-frame velocities, $\Delta\mathbf{p}_t = \mathbf{p}_t - \mathbf{p}_{t-1}$. The resulting input feature vector x_t has a dimension D_{pose} , defined by Equation 2:

$$\mathbf{x}_t = [\mathbf{p}_t \parallel \Delta\mathbf{p}_t] \in \mathbb{R}^{D_{\text{pose}}} \quad \text{with} \quad D_{\text{pose}} = 2 \times 99 = 198 \quad (2)$$

Thus, each frame t is represented by a $D_{\text{pose}}=198$ -dimensional feature vector. The batch of input sequences to the pose module, as depicted in Figure 3, therefore has dimensions $\mathbb{R}^{B \times T \times D_{\text{pose}}}$, where B is the batch size and T is the temporal window duration.

This sequence of D_{pose} -dimensional vectors is then processed by a two-layer Bi-LSTM network [24]. This network utilizes a hidden state dimension of $d_h=128$ units for each direction, and a dropout rate of 0.3 is applied between its layers. The outputs from the forward and backward passes are concatenated at each time step, yielding a feature vector of dimension $2d_h = 256$. LSTMs are particularly well-suited for this task because their gated memory cells can effectively learn long-range temporal dependencies and mitigate issues like vanishing gradients [14]. The bidirectional architecture [24] further enriches the representation at each time step by incorporating both past and future contextual information, which is crucial for accurately modeling complex pose dynamics.

The output sequence from this recurrent network, which encodes complex temporal patterns observed in the pose data, is subsequently passed through a linear projection layer (labeled as 'Fully Connected Layer' in Figure 3). This layer transforms the 256-dimensional feature vectors into $D_{\text{feat}}=128$ -dimensional vectors. This transformation involves a ReLU activation function, followed by a dropout layer with a rate of 0.3. The final output of this sub-network is the pose feature sequence, $\mathbf{F}_{\text{pose}}$, with dimensions $\mathbb{R}^{B \times T \times D_{\text{feat}}}$, as defined in Equation 3 and shown in Figure 3.

$$\mathbf{F}_{\text{pose}} \in \mathbb{R}^{B \times T \times D_{\text{feat}}} \quad \text{with} \quad D_{\text{feat}} = 128 \quad (3)$$

We incorporate frame-to-frame velocity alongside joint coordinates for four main reasons. First, it enables the model to distinguish genuine motion from static poses. Two frames with nearly identical skeleton layouts (for example, reach versus retract) can be disambiguated by the direction and magnitude of movement. Second, velocity provides rich kinematic cues that encode both speed and direction changes over time, helping the Bi-LSTM learn acceleration and deceleration phases as well as complex motion patterns. Third, when a subject remains still or moves very slowly, velocity values near zero signal “no movement,” which prevents the network from misclassifying static poses as actions. Fourth, velocity spikes or drops at action onsets and offsets, for example when speed jumps from zero to a peak and then returns to zero, create sharp temporal landmarks that make it easier for the 1-dimensional convolution heads and the RNN post-processing module to detect precise start and end boundaries.

3.3 Multi-Modal Fusion and Temporal Refinement

Having processed features from both the visual RGB stream and the pose stream, the subsequent critical step involves effectively integrating this multi-modal information. This fusion process aims to harness the complementary strengths inherent in each data modality. The features derived from the RGB stream, after the gating mechanism and a subsequent projection. Similarly, the features from the pose stream, after passing through an LSTM and a fully connected layer, are projected by into the same latent space. This initial projection of both streams ensures dimensionality compatibility for the fusion operation.

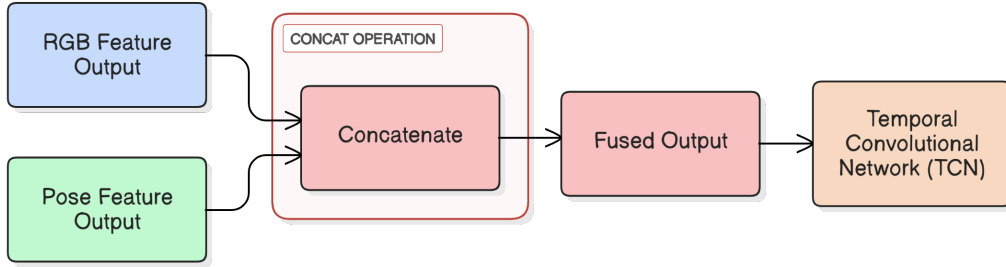


Figure 4: Multi-modal fusion module

In this study, features from the visual RGB and pose modalities, denoted as $\mathbf{F}'_{\text{rgb}}$ and $\mathbf{F}'_{\text{pose}}$ respectively, are first independently projected into a common embedding space of dimension $D_{\text{fusion}} = 512$. Following this projection, these modality-specific representations are integrated. The fusion mechanism employed involves concatenation followed by a learned convolutional transformation. Specifically, the projected RGB features $\mathbf{F}'_{\text{rgb}} \in \mathbb{R}^{B \times T \times D_{\text{fusion}}}$ and the projected pose features $\mathbf{F}'_{\text{pose}} \in \mathbb{R}^{B \times T \times D_{\text{fusion}}}$ are concatenated along their feature dimension. This operation yields a combined feature tensor, $\mathbf{F}_{\text{concat}}$ with dimensions:

$$\mathbf{F}_{\text{concat}} \in \mathbb{R}^{B \times T \times (2D_{\text{fusion}})} \quad (4)$$

Subsequently, this $(2D_{\text{fusion}})$ -dimensional concatenated tensor is processed by a 1-dimensional convolutional layer. This layer, utilizing a kernel of size 1, serves to transform the combined features back into the target D_{fusion} -dimensional space. The output of this convolutional operation is the final fused multi-modal representation, $\mathbf{F}_{\text{fused}}$, defined as:

$$\mathbf{F}_{\text{fused}} = \text{Conv1D}_{\text{kernel}=1}(\mathbf{F}_{\text{concat}}), \quad \text{where } \mathbf{F}_{\text{fused}} \in \mathbb{R}^{B \times T \times D_{\text{fusion}}} \quad (5)$$

This approach allows the network to learn an optimal combination of the concatenated multi-modal information, effectively reducing the feature dimensionality back to the common fusion space while capturing essential inter-modal relationships. Following this fusion step, the combined multi-modal features $\mathbf{F}_{\text{fused}}$ undergo a final stage of temporal refinement utilizing a TCN. While the individual streams and the fusion layer might have modeled some temporal dynamics, this post-fusion TCN is specifically designed to capture potentially longer-range temporal dependencies and complex interactions that emerge after combining the visual and pose information. This network is composed of a sequence of temporal processing blocks, each typically consisting of dilated 1-dimensional convolutions, batch normalization, ReLU activations, and dropout, as detailed by Lea et al. [17]. This architecture progressively increases the receptive field, enabling the network to further refine the temporal understanding of the fused representation. The output of this TCN module yields the final temporal features

$$\mathbf{F}_{\text{temporal}} \in \mathbb{R}^{B \times T \times D_{\text{fusion}}}, \quad (6)$$

which are then passed to the prediction heads.

3.4 Prediction heads

Based on the last fine-grained multimodal temporal features $\mathbf{F}_{\text{temporal}}$, three independent prediction heads are employed to generate the required outputs for the detection of temporal action (observed in Figure 5). Multi-head structure is common in dense prediction tasks [19]. Each prediction head is implemented as a 1-dimensional convolutional layer with 512 input channels, N_{classes} output channels, and kernel size 1, thus performing a learnable linear mapping at each time step to convert the refined temporal features into class-specific logits. The heads are as follows:

Action Classification Head: Predicts the probability distribution of the N_{classes} action categories for each time step t .

Start Boundary Detection Head: Predicts the probability for each time step t of being the starting boundary of an action instance for each class.

End Boundary Detection Head: Predicts the probability for each time step t of being the ending boundary of an action instance for each class.

The output of these heads are raw scores (logits), which are then shaped appropriately to produce the final outputs: action classification scores, start boundary scores, and end boundary scores ($\mathbb{R}^{B \times T \times N_{\text{classes}}}$).

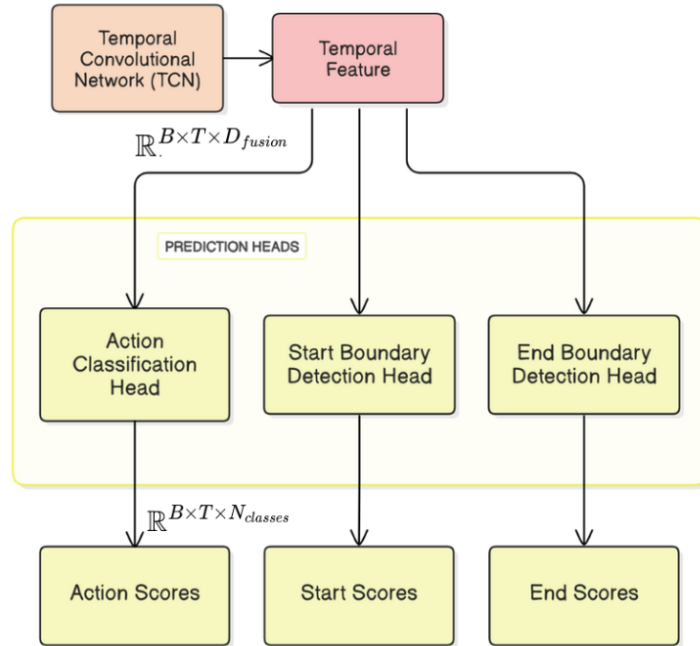


Figure 5: Diagram Showing the Prediction Heads

3.5 RNN-based Temporal Post-Processing

To enhance the frame-level predictions of the base temporal action detection model and promote temporal coherence, the system incorporates a post-processing module that is based on a RNN (in Figure 6). While able to model local temporal context in sliding windows, the base model can produce noisy or fragmented predictions while considering the entire video sequence. The RNN module endeavors to keep such issues at bay by modeling longer-

range temporal relationships on the entire video.

The input to this RNN module for a given video consists of the sequence of frame-level probability vectors ($\mathbf{p}_t$):

$$\mathbf{p}_t = [\mathbf{p}_t^{\text{action}} \parallel \mathbf{p}_t^{\text{start}} \parallel \mathbf{p}_t^{\text{end}}] \in [0, 1]^{3N_{\text{classes}}}. \quad (7)$$

Obtained after reconstructing the entire sequence by averaging the logits of all windows that contain frame t , where N_{classes} denotes the number of action classes. Specifically, for every frame t , the input vector concatenates the predicted probabilities for each action class to be active ($\mathbf{p}_t^{\text{action}}$), starting ($\mathbf{p}_t^{\text{start}}$), and ending ($\mathbf{p}_t^{\text{end}}$).

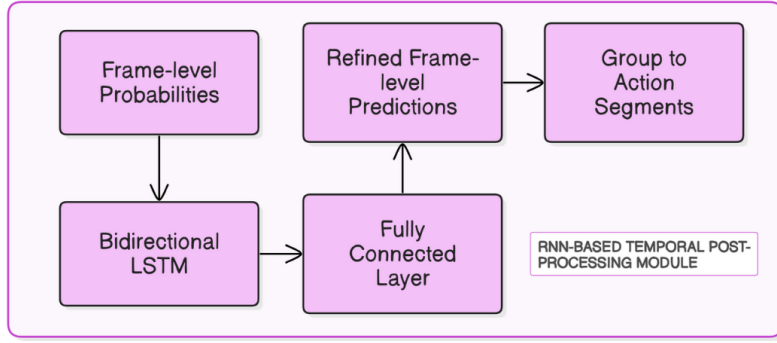


Figure 6: Overview of the RNN-Based Temporal Post-Processing Module

We employ a two-layer Bi-LSTM with 128 hidden units per direction and 0.5 dropout between layers. The bidirectional architecture allows the model to take advantage of both past and future contexts when predicting the label for the current frame. The LSTM processes the entire sequence of input probability vectors $\{\mathbf{p}_1, \mathbf{p}_2, \dots, \mathbf{p}_N\}$, where N is the total number of frames in the video. The output of each step by the LSTM is passed through a fully connected layer and then a softmax activation function that outputs a probability distribution over $N_{\text{classes}} + 1$ classes (the N_{classes} action classes and a background class). The class with the highest probability in the frame t is selected as the refined frame-level prediction $\hat{y}_t$.

Finally, the refined sequence of frame-level labels $\{\hat{y}_1, \hat{y}_2, \dots, \hat{y}_T\}$ is converted into a set of discrete action segments. Consecutive frames assigned the

same non-background action label are grouped together to form a temporal segment, defined by its start frame, end frame, and predicted action class. This process results in the final set of action predictions for the video.

4 Experiments and Results

4.1 Dataset and Experimental Setup



Figure 7: Images for Various Actions in the MERL Shopping Dataset [27]

Our experiments employ the MERL Shopping dataset [27], a controlled benchmark for fine-grained retail action detection. This corpus consists of 106 videos, each lasting approximately two minutes, captured by a fixed overhead HD camera in a lab-simulated grocery environment. Thirty-two participants each completed three shopping sessions on separate days, resulting in 60 training videos, 18 validation videos, and 28 test videos. Each frame is annotated by expert assessors with the precise onset and offset of five actions, Reach to Shelf, Retract from Shelf, Hand in Shelf, Inspect Product and Inspect Shelf (Figure 7) enabling frame-level evaluation of both classification and temporal localization.

The overhead view minimizes occlusion and preserves consistent spatial structure, simplifying tracking of objects, and discriminating between small gestures that are typically responsible for differentiating comparable actions (e.g., Reach and Retract). However, the data set experiences an extreme class imbalance (Table 1): Hand in Shelf is only 562 segments (18183 frames) during training time, compared to 1711 segments (30582 frames) for Reach to Shelf and 1621 segments (31021 frames) for Retract from Shelf. The imbalance extends to validation and test splits and threatens to bias learning towards common classes.

Table 1: Action segment counts and total durations in the MERL Shopping Dataset [27]

Action	Segments	Total Duration
Reach To Shelf	1711	30582
Retract From Shelf	1621	31021
Hand In Shelf	562	18183
Inspect Product	674	39308
Inspect Shelf	809	52011
Total	5377	171105

4.2 Frame Preprocessing and Window Sampling

In comparison to some existing pipelines that first localize the subject by aggregating pose cues in order to crop a region of interest [2, 16], our experiment retains the whole frame while preprocessing. This design preserves more context information at a greater scale and avoids potential mistakes caused by wrongful bounding-box extraction. Each raw video is separately decoded into frames, resized to 224×224 pixels, and normalized with the ImageNet standard mean and standard deviation.

Table 2: Action-duration statistics (frames)

Action Class	Min	Max	Median	Avg
Reach To Shelf	5	87	17	17.77
Retract From Shelf	5	66	18	19.14
Hand In Shelf	3	406	20	32.35
Inspect Product	4	375	41	58.32
Inspect Shelf	5	521	50	64.29

We then apply a sliding-window sampling method directly to these full-

frame sequences, sampling $T = 32$ frame length clips with 50% overlap (i.e., frames 1–32, 17–48, 33–64, ...), providing dense coverage near action boundaries and sufficient temporal context for subtle motion cues [17, 35]. This choice of $T = 32$ is further motivated by our action-duration statistics (Table 2), which show median lengths of 17–50 frames (average 17.77–64.29 frames at 15 fps). A 32-frame window effectively captures shorter average-duration actions (e.g., Reach To Shelf, Retract From Shelf, Hand In Shelf), facilitating focused feature learning, while the 50% overlap ensures adequate coverage of longer actions (e.g., Inspect Shelf) without the computational cost of a consistently larger window that could introduce redundant context for shorter events. Each output clip has shape $(C, T, H, W) = (3, 32, 224, 224)$ and is batched into (B, C, T, H, W) before being passed directly to the model for all subsequent hierarchical feature extraction and temporal modeling.

4.3 Implementation Details

4.3.1 Base Model Training

The initial phase of this experimental study involved the training of a temporal action detection model. This model architecture is designed to process pre-extracted feature representations derived from the input video sequences. Specifically, it accepts three distinct feature streams for each temporal window: features presumed to originate from a MViT, features from a ResNet-based architecture (capturing visual details), and features encoding pose-derived kinematic information. This approach allows for an investigation into how effectively action patterns can be discerned from these diverse, high-level feature sets.

The training was conducted using temporal windows of size $T = 32$ frames. The model’s parameters were optimized using the AdamW algorithm, with an initial learning rate set to 1×10^{-4} and a weight decay parameter of 1×10^{-5} . To dynamically adjust the learning rate, a ReduceLROnPlateau scheduler was employed; this scheduler reduced the learning rate by a factor of 0.1 if the validation loss failed to improve for 3 consecutive epochs. For this specific training configuration of the feature-based detector, an explicit multi-epoch learning rate warmup period was not utilized, and gradient clipping was not applied. The system was trained for a maximum of 50 epochs, employing a batch size of 256. An early stopping mechanism, based on validation loss with a patience of 10 epochs, was also implemented to prevent

overfitting.

The loss function guiding the optimization process was composed of three distinct terms, corresponding to the prediction of action presence, start boundaries, and end boundaries. Each term was calculated using a Binary Cross-Entropy with Logits loss. A significant aspect of this training regimen was the incorporation of class-specific weights into each of these three loss components. These weights were determined by calculating the inverse of the total temporal duration of each action class within the training dataset, subsequently normalized by the mean of these inverse durations. (The specific raw durations used for the 5 classes were [30582, 31021, 18183, 39308, 52011]). The individual, class-weighted loss values for action, start, and end predictions were then summed to form the total loss for optimization

$$L = L_{\text{action_weighted}} + L_{\text{start_weighted}} + L_{\text{end_weighted}}. \quad (8)$$

Explicit label smoothing was not applied in this particular training setup for the feature-based detector.

During this training process, the model checkpoint that yielded the lowest validation loss was preserved. This selected checkpoint represents the optimally trained feature-based temporal detector. The probabilistic outputs from this detector are subsequently utilized either for direct evaluation after applying post-processing techniques or as input sequences for the RNN-based temporal refinement module, which is explored in later stages of this study.

4.3.2 RNN Input Data Generation

In order to pre-process the input data for the subsequent RNN-based temporal post-processing module, a bespoke data generation pipeline was executed. This process utilized the best checkpoint obtained from the base model training procedure. To start, the pre-trained base model was utilized to perform inference on all the training and validation sets. The inference process passed the data through the same sliding window setup (32 frames) applied when training the base model, yielding raw output predictions per window.

The output of this inferential process consisted of sequences of the likelihoods of each of the types of actions as they relate to the initiation, involvement throughout the process, or closure within stated time intervals.

Therefore, for each unique video within the sets of datasets, the window level's sequences of probabilities were recast as continuous sequences covering the full length of the video. Reconstruction was achieved by summing outcomes of the aggregation of all windows over a particular frame and then averaging these results to obtain optimized and consistent metrics on the existence $\mathbf{p}_t^{\text{action}}$, the start $\mathbf{p}_t^{\text{start}}$, and end $\mathbf{p}_t^{\text{end}}$ of actions for a particular frame t within the video.

The input feature vector for the RNN at each frame t was then constructed by concatenating these three reconstructed probability vectors. Given the $N_{\text{classes}} = 5$ action classes, this resulted in an input feature dimension of $3 \times N_{\text{classes}} = 15$ for the RNN model. At the same time, the related ground truth target sequence used for training the RNN was also generated for each video. Based on the original dataset annotations, each frame within the video was assigned a single integer label of the ground truth action class that is occurring at the corresponding time step. Frames that do not fall into an annotated action segment were labeled as background labels.

Ultimately, the produced input features for the RNN, as 15-dimensional sequences, as well as the corresponding target sequences of labels, were stored systematically for each video independently in a compressed array-based form. As a result, this gave rise to unique, processed datasets optimized for later training and validation of the RNN post-processing model.

4.3.3 RNN Post-Processor Training

The final stage involved the training of the RNN post-processing module with the data generated in the previous step. The aim of this module is to enhance the action predictions at the frame level by modeling long-range temporal dependencies along the whole video sequence.

The used RNN structure was a multi-layer Bi-LSTM network. It was implemented to feed in the 15-dimensional feature vectors (retrieved by converting the concatenated base model output of start and end probabilities and action probabilities at each time step) at each time step. It had 2 LSTM layers, each 256 units size for the hidden state. The dropout with 0.5 probability was implemented between LSTM layers to avoid overfitting. The bidirection allowed the network to consider past and future context when making a prediction. The final output layer projected LSTM hidden states to logits for 6 classes (the 5 action classes, along with one additional background class).

A dedicated dataset class was employed to load preprocessed feature and label sequences stored for each video. The associated dataloader handled batching and, more importantly, applied a padding scheme to accommodate videos of varying lengths in each batch. Input feature sequences were padded with zeros, but the target label sequences were padded with a specific ignore value (-100) to ensure that these padded time steps did not contaminate the loss calculation.

The RNN model was optimized using the AdamW algorithm by applying a learning rate set at 1×10^{-3} . Training was done for a maximum of 50 iterations using a batch size of 16. The commonly used Cross-Entropy loss function was utilized by deliberately excluding the padding index (-100) from the target labels. Beyond this, a ReduceLROnPlateau learning rate scheduler was included for monitoring the validation loss for decreasing the learning rate by a factor of 0.5 if there was not any improvement over three consecutive epochs. Early stopping was also integrated into the model with a patience of 5 epochs wherein training would stop if there was not any improvement in the validation loss within this set patience time.

Throughout the training process, the model performance was also tested on the validation set following each epoch. The lowest loss validation model checkpoint was saved as the best RNN post-processor model. This final checkpoint is the fully trained temporal refinement module used for generating the final action predictions experimented.

4.4 Evaluation Metrics

To rigorously assess the performance of our experiment, we employed a selection of established metrics focusing on different aspects of detection quality. Firstly, Segment-level F1 scores were calculated at varying temporal IoU thresholds: 0.10, 0.25, and 0.50 (denoted as F1@10, F1@25, F1@50, respectively). The temporal IoU between a predicted segment $S_p = [\text{start}_p, \text{end}_p]$ and a ground-truth segment $S_{gt} = [\text{start}_{gt}, \text{end}_{gt}]$ is defined as:

$$\text{IoU}(S_p, S_{gt}) = \frac{|S_p \cap S_{gt}|}{|S_p \cup S_{gt}|} \quad (9)$$

A prediction is considered a True Positive (TP) at a given threshold θ if its IoU with a ground-truth segment of the same class is greater than or

equal to θ , and that ground-truth segment has not already been matched with a higher-confidence prediction. Precision (P) and Recall (R) are then calculated based on the counts of TPs, False Positives (FP – predicted segments not matching any ground truth sufficiently), and False Negatives (FN – ground-truth segments not matched by any prediction):

$$P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}. \quad (10)$$

The F1 score, representing the harmonic mean of Precision and Recall, is computed as:

$$F1 = \frac{2PR}{P + R}. \quad (11)$$

Reporting F1 at multiple IoU thresholds allows for characterizing detection quality from coarse localization (IoU=0.10) to stricter boundary alignment (IoU=0.50).

Secondly, we utilized the mAP@mid metric. This metric relaxes the strict boundary overlap requirement of IoU. A prediction S_p is considered a True Positive for a ground-truth segment S_{gt} of the same class if the temporal midpoint of the prediction,

$$\text{mid}_p = \frac{\text{start}_p + \text{end}_p}{2}, \quad (12)$$

falls within the ground-truth interval, i.e., $\text{start}_{gt} \leq \text{mid}_p < \text{end}_{gt}$ (assuming appropriate handling of multiple matches based on confidence scores). AP is then calculated for each class based on the precision-recall curve generated using this midpoint criterion for matching. The final mAP@mid is the mean of these AP values across all action classes, providing a measure focused on the core recognition aspect rather than precise boundary delineation. The AP for a class c is calculated as:

$$\text{AP}_c = \sum_k P_c(k) \Delta R_c(k), \quad (13)$$

where predictions are ranked by confidence, $P_c(k)$ is the precision at rank k , and $\Delta R_c(k)$ is the change in recall at rank k . Then the mAP@mid is calculated as:

$$\text{mAP@mid} = \frac{1}{C} \sum_{c=1}^C \text{AP}_c,$$

where C is the number of classes.

Thirdly, Frame-wise Accuracy was computed to assess the per-frame classification correctness across the entire dataset after merging predictions from sliding windows. It represents the proportion of frames for which the predicted action label (or lack thereof) correctly matches the ground-truth label. Letting T_{total} be the total number of frames evaluated, pred_t the predicted label for frame t , and gt_t the ground-truth label for frame t , the accuracy is:

$$\text{Accuracy} = \frac{1}{T_{\text{total}}} \sum_{t=1}^{T_{\text{total}}} \mathbf{1}(\text{pred}_t = \text{gt}_t), \quad (14)$$

where $\mathbf{1}(\cdot)$ is the indicator function (evaluating to 1 if the condition is true, 0 otherwise). This metric provides a global view of classification performance at the most granular level. All evaluations were performed on the held-out validation or test splits as specified, ensuring that the results reflect the model’s generalization capability on unseen data and maintaining strict subject independence between training and evaluation sets.

4.5 Evaluation of Architectural Components and Fusion Strategies

In this section, we conduct a series of experiments to evaluate the impact of a critical design choice within the model: the final fusion method employed to integrate the aggregated RGB feature stream and the Pose feature stream. To ascertain a more effective approach for this integration, two prevalent mechanisms were investigated: feature concatenation followed by a convolutional transformation, and a learned weighted summation. For this comparative study, both streams (RGB and Pose) are assumed to have been processed and projected to a common feature dimension (e.g., 512), prior to this final fusion step.

The first configuration, termed the Concatenation-based Fusion (Baseline Model), involves concatenating the processed aggregated RGB features and Pose features (each projected to a compatible dimension) along their feature dimension. This results in a feature tensor with a combined dimensionality. A 1-dimensional convolutional layer, typically with a kernel size of 1, is then applied immediately after concatenation. The role of this convolutional layer is to learn a transformation of the concatenated features and to adjust the channel dimensionality back to the target fusion space of 512. The output is subsequently passed to a TCN block for further temporal modeling.

The second architectural variant, termed the Weighted Summation Fusion (SumFusion Model), replaces the direct concatenation and the subsequent fusion convolution layer with a learned weighted sum operation. Specifically, the aggregated RGB features and Pose features, both having been projected to the same fusion dimension of 512, are combined by calculating a weighted sum. In this process, the system learns scalar weights that determine the contribution of each feature stream (RGB and Pose) to the final fused representation. These weights are typically normalized (for instance, using a softmax function applied to underlying learnable parameters) to ensure they represent the adaptive, relative importance of each modality. The resulting fused features, also in the 512-dimensional space, are then passed to the TCN block. The comparative performance results of these two fusion methodologies are presented in Table 3

Table 3: Comparative performance results over three experiments

Methods	F1@10,25,50	mAP@mid	Accuracy
CONCAT	81.8, 76.5, 63.9	93.1	88.6
SUMFUSION	83.2 , 77.4, 65.5	92.8	88.8
CONCAT+TCN	82.5, 78.1 , 65.2	95.7	88.4

4.6 Results and Analysis

Following the insights gained from the validation set, the system configuration incorporating the RNN-based temporal post-processing module was selected for final evaluation on the held-out test set. The results were then contextualized by comparing them against previously published state-of-the-art methods on the MERL Shopping dataset [27]. Table 4 presents these benchmark comparisons. The system investigated in this study ("Our experiment" achieved a mAP@midpoint of 95.7% and a frame-level accuracy of 88.4% on the test set. For segment-level F1 scores, the system obtained F1@10 of 82.5%, F1@25 of 78.1%, and F1@50 of 65.2%.

Analysis of these results (Table 4) reveals that the investigated system demonstrates substantially superior performance in terms of mAP@midpoint and frame-level accuracy when compared to all previously reported methods

Table 4: Benchmark comparison on the MERL Shopping Dataset

Methods	F1@10,25,50	mAP@mid	Accuracy
MSN Det [27]	46.4, 42.6, 25.6	81.9	64.6
MSN Seg [27]	80.0, 78.3, 65.1	69.8	76.3
Dilated TCN [17]	79.9, 78.0, 67.5	75.6	76.4
ED-TCN [17]	86.7, 85.1, 72.9	74.4	79.0
HA-Net [11]	82.5, 78.1, 71.2	76.7	79.3
Our experiment	82.5, 78.1, 65.2	95.7	88.4

on this dataset. This is a key learning, indicating the system’s strong capability in accurately identifying the central temporal aspect of an action and in correct frame-level classification. However, a further learning point emerges from the IoU-based F1-scores. While the RNN module significantly improved these scores compared to the Heuristic Baseline, the F1@50 score of 65.2% on the test set does not yet reach the performance levels of some leading approaches specifically optimized for boundary precision, such as ED-TCN (72.9%) [17] or HA-Net (71.2%) [11]. This suggests that while the system excels at core action recognition and classification, further focused investigation could enhance the precise localization of action start and end boundaries to achieve higher temporal overlap under stricter IoU criteria. The observed balance of performance might stem from the study’s implicit optimization focus or the RNN’s primary role in temporal smoothing rather than explicit boundary regression for IoU maximization.

4.7 Discussion

The experimental results from this study offer some insights into the performance of the evaluated two-stage approach for temporal action detection, with a particular focus on the potential impact of an RNN-based Post-Processing module. The system configuration explored in this experiment, which integrated this RNN module, yielded a mAP@midpoint of 95.7%, a frame-level accuracy of 88.4%, and an F1@50 score of 65.2% on the MERL

Shopping [27] test set. Qualitative observations suggest that the RNN module appeared to help in smoothing predictions and addressing some fragmented detections that can arise from a window-based base model.

The observed performance levels, particularly in mAP@midpoint and frame-level accuracy, may be related to the RNN module’s capacity for modeling long-range temporal dependencies. As detailed in the Methodology section, the base model processes inputs in fixed-size windows, which can sometimes result in noisy or inconsistent predictions when viewed globally. It is thought that the Bi-LSTM network in the post-processing module, by operating on reconstructed probability sequences from the entire video, could capture broader temporal context and integrate information across distant frames. This, in turn, might allow it to filter some spurious frame-level predictions and merge fragmented segments, potentially contributing to more temporally coherent outputs and the recorded mAP@midpoint and accuracy figures.

Nevertheless, when comparing the results with established state-of-the-art methods, it’s evident that there are areas for further exploration and potential improvement. While the experimented approach showed encouraging results in midpoint localization (mAP@midpoint 95.7%) and frame-level accuracy (88.4%) relative to some published figures on the MERL Shopping dataset (Table 4), its F1@50 score (65.2%) did not reach the levels of some specialized methods like ED-TCN (72.9%) [17] or HA-Net (71.2%) [11]. The strong performance in mAP@midpoint, which emphasizes correct action classification around the temporal center of an event by relaxing strict boundary overlap requirements, suggests a robust classification capability within the system. This effective classification might be partly attributed to the expressive power of the MVIT backbone used for global spatio-temporal feature extraction in the RGB stream, which is designed to capture both local and global information in video clips through its hierarchical multiscale attention mechanism. However, the discrepancy with F1@50 scores also suggests that while the overall classification and identification of core action segments may be strong (potentially enhanced by the RNN module), enhancing the precise localization of segment start and end boundaries for improved temporal overlap could be a focus for future study.

During the base model training, regularization techniques such as label smoothing (with a factor of 0.1) were consistently applied to prevent overconfidence and improve generalization. It is also worth noting, as is common with deep learning models, that the performance observed in this experiment

could be sensitive to the specific hyperparameter choices made for both the base model and the RNN module. The two-stage nature of the examined architecture also introduces a level of complexity that could be compared against potential end-to-end solutions. For future learning and exploration, several avenues could be considered based on the observations from this experiment. Investigating methods to potentially improve boundary prediction accuracy within the RNN framework, perhaps through auxiliary boundary regression tasks or specialized loss functions focusing on IoU, might help in addressing the gap in F1-scores. Exploring end-to-end trainable architectures that integrate temporal modeling more directly with feature extraction could be an interesting direction, potentially simplifying the pipeline by removing the intermediate probability reconstruction step. Additionally, experimenting with different recurrent architectures or incorporating attention mechanisms within the post-processing module could yield further insights. Validating the experimental setup on diverse datasets would also be valuable for understanding its characteristics more broadly. Lastly, a deeper investigation into the trade-offs between optimizing for midpoint accuracy versus IoU-based metrics could provide useful learnings for tailoring such systems to specific application needs.

5 Conclusion

In this experimental study, we explored the problem of temporal action detection in retail environments by assessing a two-stage architecture that combined multi-modal feature extraction with temporal refinement. The first stage involved a base model designed to process RGB and pose-velocity inputs within sliding temporal windows, generating initial frame-level predictions for action class, start, and end probabilities. To address potential fragmentation and enhance temporal consistency, a second-stage post-processing module, featuring a Bi-LSTM, was experimented with to learn longer-term temporal relationships across entire video sequences.

Evaluations conducted on the MERL Shopping dataset indicated the potential of this architectural setup. The system, incorporating the RNN post-processing step, achieved notable segmentation metrics such as mAP@midpoint (95.7%) and frame-level accuracy (88.4%). These results suggest the system’s capability in capturing temporal boundaries and frame-wise action semantics with a degree of accuracy.

Despite these encouraging observations, our experiments also highlighted areas for further investigation. For instance, while the system performed well in terms of mAP@midpoint and frame-level accuracy, the precision of segment boundaries, as measured by F1@50 (65.2%), did not match that of some specialized boundary-focused architectures like ED-TCN [17] and HA-Net [11]. This suggests that while the RNN module contributes to overall coherence, further work could explore combined architectures or more specialized boundary regression mechanisms to potentially enhance segment delineation.

In general, this experimental study tends to support the idea that employing a learned recurrent network for post-processing can be a useful step for improving the temporal coherence and accuracy of predictions from a window-based action detector. The explored RNN post-processing module seems to offer a promising approach for enhancing temporal action segmentation quality, particularly in contexts where precise frame-level understanding and the localization of the central aspects of actions are important.

References

- [1] Bertasius, G., Wang, H., Torresani, L.: Is space-time attention all you need for video understanding? In: International Conference on Machine Learning (2021)
- [2] Cao, Z., Hidalgo, G., Simon, T., Wei, S.E., Sheikh, Y.: Openpose: Real-time multi-person 2d pose estimation using part affinity fields. *IEEE Transactions on Pattern Analysis and Machine Intelligence* **43**(1), 172–186 (2019)
- [3] Carreira, J., Zisserman, A.: Quo Vadis, Action Recognition? A New Model and the Kinetics Dataset. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). pp. 4724–4733 (2017). <https://doi.org/10.1109/CVPR.2017.502>
- [4] Chain Store Age: Survey: 63% of retailers call ai ‘critical’ to maintaining a competitive edge. Chain Store Age, March 2025, <https://chainstoreage.com/survey-63-retailers-call-ai-critical-competition>
- [5] Countouris, C.: Ai gets real for retailers: 9 out of 10 retailers now adopting or piloting ai, latest nvidia survey finds.

- NVIDIA Blog, January 10, 2025, <https://blogs.nvidia.com/blog/ai-in-retail-cpg-survey-2025/>
- [6] Dell Technologies: Accelerate intelligent retail with dell’s simplified retail edge. April 21, 2022, <https://www.dell.com/en-us/blog/accelerate-intelligent-retail-with-dells-simplified-retail-edge/>
 - [7] Fan, H., Xiong, B., Mangalam, K., Li, Y., Yan, Z., Malik, J., Feichtenhofer, C.: Multiscale vision transformers. In: International Conference on Computer Vision (ICCV) (2021)
 - [8] Farha, Y., Gall, J.: Ms-ten: Multi-stage temporal convolutional network for action segmentation. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) (2019)
 - [9] Feichtenhofer, C., Fan, H., Malik, J., He, K.: Slowfast networks for video recognition. In: Proceedings of the IEEE International Conference on Computer Vision (ICCV) (2019)
 - [10] Financial Times: Amazon bets savings from automation can help fuel ai spending boom. Financial Times, February 25, 2025, <https://www.ft.com/content/50b7ecc3-08de-433a-9a5b-6d6590cf8179>
 - [11] Gammulle, H., Denman, S., Sridharan, S., Fookes, C.: Hierarchical attention network for action segmentation. arXiv preprint arXiv:2005.03209 (2020)
 - [12] GroceryDoppio, National Retail Federation: Walmart’s ai initiatives have cut inventory shrinkage by 15%. GroceryDoppio Article, October 2024, <https://www.grocerydoppio.com/articles/groceries-leading-the-autonomous-checkout-revolution>
 - [13] He, K., Zhang, X., Ren, S., Sun, J.: Deep residual learning for image recognition. In: IEEE Conference on Computer Vision and Pattern Recognition (CVPR) (2016)
 - [14] Hochreiter, S., Schmidhuber, J.: Long short-term memory. *Neural Computation* **9**(8), 1735–1780 (1997)
 - [15] Ishikawa, K., Sadeghian, H., Thakoor, A., Shah, N.: Asrf: Action segment refinement framework for precise video segmentation. In: Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV) (2021)

- [16] Kazemi, M., Kamali, T., Moaddeli, S., Mahoor, M.H.: Using partial pose in action detection for the merl shopping dataset. In: IEEE Winter Conference on Applications of Computer Vision (WACV) (2019)
- [17] Lea, C., Flynn, M.D., Vidal, R., Reiter, A., Hager, G.D.: Temporal convolutional networks for action segmentation and detection. In: IEEE Conference on Computer Vision and Pattern Recognition (CVPR) (2017)
- [18] Li, L., Xu, X., Zhou, S.: Ms-tcn++: Multi-stage temporal convolutional network with dual dilated layers. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)* (2020)
- [19] Lin, T., Goyal, P., Girshick, R., He, K., Dollár, P.: Focal loss for dense object detection. In: *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. pp. 2980–2988 (2017)
- [20] Lugosch, L., et al.: Mediapipe: A framework for building perception pipelines. In: *ACM/IEEE International Conference on Human-Robot Interaction (HRI)* (2019)
- [21] Moghaddam, A., Liu, S., Porikli, F.: Follow the attention: Joint activity recognition and object interaction in shopping videos. *arXiv preprint arXiv:1906.XXXX* (2019)
- [22] Pantano, E., Vannucci, V.: Who is innovating? an exploratory research of digital technologies diffusion in retail industry. *Journal of Retailing and Consumer Services* **49**, 297–304 (2019)
- [23] Paolanti, P., Gambi, M., Landi, C., Furfari, F.: Vrai: Virtual retail assistant interface using overhead rgb-d streams. In: *Proceedings of the International Conference on Machine Vision Applications (MVA)* (2020)
- [24] Schuster, M., Paliwal, K.K.: Bidirectional recurrent neural networks. *IEEE Transactions on Signal Processing* **45**(11), 2673–2681 (1997)
- [25] Shah, M., Lee, B.H., Kim, S.: Pose-augmented cnns with late fusion for fine-grained action recognition. In: *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)* (2022)
- [26] Simonyan, K., Zisserman, A.: Two-stream convolutional networks for action recognition in videos. In: *Advances in Neural Information Processing Systems (NeurIPS)* (2014)

- [27] Singh, B., et al.: A multi-stream bi-directional recurrent neural network for fine-grained action detection. In: IEEE Conference on Computer Vision and Pattern Recognition (CVPR) (2016)
- [28] Steven, K.: 30+ fascinating ai in retail statistics & trends for 2025. KhrisDigital, January 12, 2025, <https://khrisdigital.com/ai-in-retail-statistics/>
- [29] Szegedy, C., Liu, W., Jia, Y., Sermanet, P., Reed, S., Anguelov, D., Erhan, D., Vanhoucke, V., Rabinovich, A.: Going deeper with convolutions. In: IEEE Conference on Computer Vision and Pattern Recognition (CVPR) (2015)
- [30] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I.: Attention is all you need. In: Advances in Neural Information Processing Systems (NeurIPS) (2017)
- [31] Weinzaepfel, P., et al.: Towards weakly-supervised action localization. IEEE Transactions on Pattern Analysis and Machine Intelligence **41**(5), 1119–1131 (2019)
- [32] Wen, X., Chen, J., Lu, H.: Robust retail behavior recognition under challenging lighting conditions using deep video models. Sensors **22** (2022)
- [33] Yi, Y., Lee, J., Oh, S.: Asformer: Action segmenter with local transformer for fine-grained video segmentation. In: Proceedings of the British Machine Vision Conference (BMVC) (2021)
- [34] Zhao, Q., Tang, Y., Li, J.: Amfi-net: Attention-based multimodal feature integration for human action recognition. Journal of Visual Communication and Image Representation (JVCIR) (2025)
- [35] Zhao, Y., et al.: Temporal action localization: Recent advances and future challenges. Pattern Recognition **112** (2021)